

ESP32 Peer to Peer Smart Home System

Spring 2025 ECE 56800 Project Final Report: ESP Smart Home

Ian Searle (Lead), John Gahnz, Alex Ouellette, Johanan Agarwal, Eugene H. Wang, Majharul Hoque

ABSTRACT

This project describes a custom ESP32-based peer-to-peer smart home automation and monitoring system designed to operate without cloud connectivity or a central hub. Existing smart home solutions suffer from vendor lock-in, planned obsolescence, and reliance on external services that can fail during internet outages. To address these limitations, a system was designed using a mesh network of ESP32[6] devices capable of direct device-to-device communication, local computation, and secure enrollment. The system leverages the ESP-NOW protocol, which allows for a custom message package. The proposed version of the custom message includes sender health data, action flags, and a list of past nodes. The message format is designed to relay sensor data across nodes, with a host PC handling data logging and system management, but can operate without the PC. Results show that reliable multi-hop communication, sensor data transmission, and host-side logging are achievable using this architecture, with some remaining limitations being incomplete such as the self-healing network behavior and user interface. This work demonstrates that a low-cost, open-source alternative to proprietary smart home platforms is achievable using embedded hardware alone.

1 INTRODUCTION

Smart home systems have become increasingly common as consumers seek to secure and enhance their homes, yet the majority of current consumer solutions depend on closed platforms, vendor-specific ecosystems, and persistent cloud connectivity. When internet services are disrupted or vendors discontinue support, these systems can become entirely inoperable — leaving functioning hardware as little more than e-waste. Local alternatives exist but introduce their own problems, including weaker security during device enrollment and growing network congestion as deployments scale. Another point against a local alternative is it requires more setup and complicated maintenance. This project addresses that gap by designing and prototyping an ESP32-based peer-to-peer mesh network that enables direct device communication, local computation, and secure user-guided enrollment — all without relying on a central hub or external server. A central hub is used to allow the user to interact with the private mesh network. Although the version presented in this report is a prototype, the focus is on the technological and cultural impact of the project.

2 BACKGROUND

The concept for this project originated with team lead Ian Searle, who had been developing the idea prior to this course and brought both the motivation and the initial system architecture to the team. Ian's prior work focused on how to design and implement a peer-to-peer embedded system for home automation, and that foundation gave the team a significant head start in defining the problem space and proposed solution.

The broader smart home market addressed by this project is currently dominated by a small number of large platforms, including Google Home[8], Amazon Alexa[2], and Apple HomeKit[5]. These platforms typically route device communication through company-managed cloud infrastructure. While these systems offer convenience and broad device compatibility, they remain dependent on continued vendor operation, service availability, and long-term platform support. When companies discontinue support or experience service outages, otherwise functional hardware can become unusable, contributing to electronic waste as smart home adoption continues to increase.

Locally-run alternatives such as Home Assistant reduce cloud dependency but still route all traffic over the home's Wi-Fi network, creating bandwidth congestion at scale and relying on enrollment methods that can expose devices to spoofing or unauthorized access. Home Assistant also requires the user to "tinker" to build the system and to have a more advanced level of technical knowledge. These increased barriers of entry, can make it difficult for people seeking a fully private or long term smart home system to achieve their goal. The ESP32 micro controller [6], developed by Espressif Systems, offers a low-cost platform with built-in Wi-Fi capabilities that has seen growing adoption in IoT applications — making it a natural fit for a custom, locally-controlled mesh system. Ian's prior thinking on system design and implementation directly shaped the team's decision to pursue this architecture rather than building on top of an existing framework.

3 PROBLEM STATEMENT

The core challenge of this project was to design a system that can evolve into a user-friendly solution while still meeting all functional, security, and scalability requirements. This challenge extends beyond simply connecting independently functioning ESP32 sensor nodes into a peer-to-peer network.

To be considered successful, the system was required to satisfy several key design requirements. From a functional standpoint, the system must support reliable multi-hop communication between nodes, transmit sensor data accurately across the mesh, and operate without reliance on a central hub or external server. The host PC must be able to log data and issue commands, but the network must remain operational without it. From a security standpoint, the enrollment process needed to prevent untrusted devices from joining the network. Although full encryption was outside the primary scope of this prototype, the system was required to reduce exposure to remote internet-based attacks and require physical proximity for network access. From a scalability perspective, the message-packet format needed to be flexible enough to support additional sensors and node types without requiring a redesign of the communication layer. The software architecture also needed to remain modular so that new sensors could be integrated without disrupting existing network behavior.

Meeting these requirements within the constraints of embedded hardware, remote collaboration, and a single-semester development timeline represented the central engineering challenge of this project.

4 PROPOSED APPROACH

The implemented approach is based on an ESP-NOW mesh network in which messages carry both sensor payloads and minimal routing information, enabling multi-hop communication without requiring a central hub. The system was designed around a defined packet structure that includes sender health data, action flags, and a “past nodes” list). A host PC was used primarily for data logging, command management, and system monitoring, while the embedded network was designed to remain operational without continuous host availability.

Each major system component is evaluated individually in later sections. Task assignments of the project is discussed in greater detail in Section 10. Each sensor node was assigned to a primary engineer, while mesh-network communication, GUI, hardware diagrams, and documentation were developed collaboratively by multiple team members.

4.1 Message Packet

The ESPNow protocol works within a total message packet size of 250 bytes.[7] This packet size allows for a great deal of customization for different use cases. Table1 shows how the message packet is divided between different sections for the system to monitor its status and messages.

Bytes	Content
0–5	Destination MAC address (6 bytes)
6–11	Sender MAC address (6 bytes)
12	Action (1 byte)
13	Message length (1 byte)
14–45	Message payload (32 bytes, zero-padded if shorter than 32)
46	Flags (1 byte)
47–56	Health report (10 bytes, all zeros if absent)
57–66	Hop trail (10 bytes, 0x00 indicates empty slot)

Table 1: Custom message packet format used for ESP32 peer-to-peer communication.

4.1.1 MAC Addresses. Each message packet will include two MAC addresses, the sender and the destination. These are included for broadcasting function. Each hop node will be able to know which direction the packet is traveling according to the hop order. Each subsequent node will then be able to send the packet toward the destination node.

4.1.2 Action Flags. Since each message can serve multiple purposes, a single byte will be included to represent what actions each message is serving. Table 2 contains the breakdown of each flag.

Flag Name	Hex	Binary	Description
ACT_SENSOR_RPT	0x01	0000_0001	Sensor reporting data value
ACT_TEST	0x02	0000_0010	Test communication / link check
ACT_RPT_ACTION	0x04	0000_0100	Reporting completed action
ACT_REQ_ACTION	0x08	0000_1000	Requesting action at destination
ACT_ADD_PEER	0x10	0001_0000	New peer addition request
ACT_SYNC_PEERS	0x40	0100_0000	Full peer map synchronization
ACT_REPORT_HOME	0x80	1000_0000	Forward message to host

Table 2: Action byte flag definitions.

Each flag can be OR-ed together to show all the different functions of the message packet. These flags can then be decoded to accomplish the correct actions by the receiver.

4.1.3 Message and Message Content: 32 bytes of the packet will be dedicated for each sensor’s message. 32 bytes was chosen to have enough space that every sensor should be able to send its information but also not too long, and can be expanded if needed. Within the current setup there is no standardized messaging structure, so each sensor can create its own style of message. Due to remote nature of this project, it was decided each sensor defines its own message structure. Each message packet also includes the total length of the message. By including the message length, each device can know what each the whole message is and not mistake any of the zeros for a part of the message. Logging will currently logs the whole message.

4.1.4 Health Report: The health report is a fixed **10-byte field** inside each ESP-NOW message packet. it is made by the MAX17048 Battery Module [3]. It gives the host node a quick summary of the sensor node’s operating condition without using the 32-byte sensor message payload.

Bytes	Content
0	Temperature, signed integer value in Celsius
1	Battery state of charge, 0–100%
2–5	Uptime, node runtime in seconds
6–9	Reserved, zero-filled for future expansion

Table 3: Health report byte layout.

Example decoded health output:

```
{'battery': 97, 'uptime': 12903, 'temp': 32}
```

4.1.5 Hop Trail. The Hop Trail represents the portion of the message of how many nodes the message has traversed through. Instead of putting the whole MAC, the ID for each node is used. Each time the message goes through a node it can store up to 10 hops, at which point the message will be considered lost.

4.2 Code Design

The overall idea for the design for the code was to build libraries in Python and import the library to create a modular style of code. This also allows for each sensor to be built and tested and easily be converted to work with the other files.

The current approach focuses on building a functional prototype, leading to a more modular design. Using multiple libraries that can be imported, each sensor's code will be based on the complexity of the individual sensor and then use a shared ESP-NOW protocol. A standard abstraction layer was not created since each sensor is unique in how the ESP interacts with it. This was chosen to allow for speed of development but will be required for long term development and maintenance.

4.2.1 ESP-NOW Communication Wrapper. Using the library for ESP-NOW is not enough to achieve the project. A library that sets up the ESP-NOW communication protocol, handles encoding and decoding of messages, and is shared across all sensor nodes [15].

4.2.2 Configuration Files. Each device will contain three configuration files, each storing different data. A JSON configuration file contains the device's name, hop value, and ID. A Python configuration file contains variable names for pins, constants, and other code-specific values. Finally, a peer file contains the names of the device's nearest peers and their peers, allowing each device to store only the information needed for message construction and correct packet routing.

4.3 Sensor Choice

Sensors were chosen based on low cost and availability, with a target total device integration cost of around 10 U.S. Dollars. The chosen sensors are listed in table 4. A stretch goal was also established to pursue package or front door object detection.

Sensor Name	Sensor Model	Owner
Air Quality	SGP40 [13]	Eugene
Light Detection	BH1750 [14]	Alex
Motion Sensor	Adafruit PIR [9]	Alex
Temperature and Humidity	DHT20 [4]	Alex
Leak Detection	Liquid Level 4965 [1]	Johanan
Front Door Watcher	Raspberry Pi + USB Camera	Johanan

Table 4: Proposed Sensors

4.4 Home/ Host PC

While a central hub has been cited as a drawback of current smart home solutions, it was determined that a host access point would still be required for this system. Although the network is capable of operating without the host PC, some functionality is dependent on its connection. Due to the peer-to-peer nature of ESP-NOW, an ESP32 host node is required for network updates and user commands to be issued. When connected to a host PC, the host node serves as the access point for the user interface, as well as a point for data to be logged for long term storage and reference.

4.5 Security

None of the participants on this project have a strong background in cyber security, so the security design focused on laying down the foundation for more advanced security. The current enrollment system requires a knowledge of what the MAC is and requires it be input manually. See 4.5.1 for more details about how current enrollment system works.

ESP-NOW does offer an encrypted mode, however it was decided to not use it for simple functionality at this point. While the network is not safe from some attacks, it is safe from remote internet attacks and attackers would need to be physically located on site to access the network. An attacker would need to be able to copy and make the same MAC addresses in order to connect to the network. Adding encryption and security keys would also not be difficult to add.

4.5.1 Enrollment. The process to enroll a new sensor should not leave the network open to non trusted sources. the proposed enrollment system will require a manual input of the device ID. The user will need to input information about the new node. This can be done via the logging/GUI or via direct editing of the files. If time allows automatic enrollment will also be added.

4.6 Mesh Network

The ESP smart home will be designed to deliver packets of data between devices in the mesh network. Nodes will use the best possible route to the host for information to be presented on the GUI. In addition, the system will be self-healing to allow the network to perform adequately even when a node loses its connection. With the self-healing, the system will utilize loop prevention techniques to limit the number of hops between nodes. This will optimize performance in the network, allowing a better final product.

5 EVALUATION

The system was evaluated against five criteria defined in the original project proposal, as shown in 5: sensor implementation, mesh network functionality, message format compliance, system management, and host PC integration. Each category was assessed through targeted testing designed to produce measurable, reproducible results. The mesh network and sensor subsystems performed as expected, with a functional GUI demonstrated. The GUI was simplified to maintain focus on the embedded systems aspects of the project. The mesh network was completed with the exception of the self-healing aspect, the reasons for which are discussed in later sections along with proposed future improvements. The following subsections describe the methodology and results for each component in turn.

5.1 Sensors

All sensors were successfully implemented and validated within the scope of the project. Each sensor was built at a relatively low cost, with an average total cost of around 10 USD including a battery. The light, motion, and temperature sensors were combined into a single unit referred to as Room Detection. All sensors were confirmed to be compatible with the MAX17048 Battery Module, which was selected to monitor battery health status across all nodes. Temperature readings from the DHT20 were validated against a

Section	Evaluation Criteria	Result
Sensors	ESP Based, Low cost, monitoring, documented	Pass
Mesh Network	Packet communication, loop prevention	Pass
	Self Healing	Partial
Message Format	Fit 250 Byte Limit, contain all needed data	Pass
System Management	Sensor enrollment, settings, GUI	Pass
Host PC	Logging of Data, system management access	Pass

Table 5: Evaluation criteria and ratings for the proposed smart home system.

reference thermometer and found to be within 0.5°C, confirming acceptable accuracy for a home monitoring application.

5.2 Mesh Network

The mesh network was validated across a five-node configuration during final integration testing, with all nodes successfully transmitting sensor data to the host. The network functioned as designed with the exception of auto mapping, due to difficulty obtaining consistently weak signal conditions. Due to the small size of the test apartments, it was difficult to get consistently weak signals. The ESP32s could cover around 1000 square feet from the device with their radio. With this constraint, loop prevention and device mapping were set aside.

5.2.1 Code Snippets. Here is the configuration file for the host for the ESP-NOW communication wrapper. The assigned name, hope value, and ID values are contained in this file.

```

1 {
2   "name": "host",
3   "hop": 0,
4   "id" : 1
5 }
```

Below is the peer file. In the example, the leak sensor is the listed peer with its peers under the neighbors. Each peer is then added in series following the same format

```

1 {
2   "peers": {
3     "leak_sensor": {
4       "mac": "B8:F8:62:D5:CC:A4",
5       "neighbors": ["host", "air_quality"],
6       "hop": 1,
7       "id": 11
8     }, ...
9   }
10 }
```

5.3 Self-Healing

Self-Healing was a stretch goal taken on by the team. The team tried to utilize Dijkstra's algorithm, an algorithm for finding the shortest

path between nodes in a weighted graph. Unfortunately, the ESP board had too strong of an antenna for a 1000 square foot apartment. This meant that every node in the apartment had a "most optimal path" directly to the host. To demonstrate self-healing, the team created an artificial path that looked like "NODE A -> NODE B -> HOST". Upon removing Node B, the path looked like "NODE A -> HOST".

The team used three ESP32 boards to mimic the paths above. The implementation of self-healing was coded to adapt based on whether Node B was plugged in. Node A would look for Node B. If it wasn't there, or it disappeared, then Node A would connect directly to the host.

This led the team to debug using a color scheme provided in 6. Upon initialization, Node B was connected directly to the Host and showed a Green LED. Node A was connected to Node B and showed a Cyan LED. When information would transfer, Node A would become Purple, showing the data had been received. After running for several minutes, Node B was unplugged. This caused Node A's LED to become Yellow for a brief second, before connecting directly to the host and turning green.

Color	Indication
Green	Connected directly to host
Cyan	Routing through node
Purple	Receiving information from node
Yellow	Finding best path
Blue	Disconnected from network

Table 6: ESP Node Color Key.

This led to great results toward a self-healing network. Unfortunately, data packets were being discarded by the host when Node B was unplugged. Without correct data transfer, the team decided to discard the functionality to pursue a working system. Proposed solutions will be discussed later in the paper.

5.4 Camera Detection Pipeline

The Raspberry Pi runs a two-stage detection pipeline to stay within the constraints of a Pi 3 running on CPU. A lightweight frame-differencing algorithm first checks for motion by comparing a running weighted average against the current frame; only when motion exceeds a minimum contour area is the more expensive neural network invoked. The model is a MobileNet SSD [10], loaded through OpenCV's DNN module and run entirely on CPU at 320×240 resolution and 5 FPS. When a person is detected above the confidence threshold, the score is sent via UDP to the camera bridge ESP32, which encodes it into the standard ESP-NOW packet for relay across the mesh. Table 7 summarizes pipeline parameters.

Component	Detail
Transport (RPI to ESP32)	UDP on local network
UDP payload	{confidence: 0.87}
ESP-NOW message	CAM:PERSON:<score>
Confidence threshold	0.50

Table 7: Camera detection pipeline parameters.

5.5 Message Format

The custom message packet format successfully meets all design requirements, coming in at 67 bytes — well under the ESP-NOW 250-byte limit — while carrying all necessary routing, action, health, and sensor data. The implementation proved reliable across direct and multi-hop communication during testing. The packet as outlined in 1 contains enough space for every point of information, and the 32-byte message payload provides sufficient space for all sensor reports. The MAX17048 fills out all designated health information for the packet.

5.6 System Management

System management was handled through a GUI that provided the user with a central point of control for the mesh network. The GUI displayed a live data feed of incoming sensor readings and provided alert notifications when sensor values fell outside of expected ranges. Users were also able to adjust sensor settings and logging options directly through the interface without needing to manually edit configuration files. While the GUI was simplified compared to the original proposal to keep focus on the embedded aspects of the project, it demonstrated the core system management functionality required for evaluation.

5.7 Host PC

The host PC served as the primary access point for data logging and system management through the GUI. Incoming sensor data received by the host node was passed to the GUI and simultaneously saved to a CSV file, with each entry containing a timestamp and the associated message information. The system was run overnight without any errors or packet loss observed, demonstrating stable long-term operation. The host node could be disconnected without disrupting communication between nodes, with the only loss being access to the GUI and the CSV logging functionality.

6 DISCUSSION

6.0.1 Strengths and Weaknesses. Low-cost sensors were successfully utilized to develop a smart home with a comprehensive mesh network. A key part of the project's success was the delegation of one sensor per developer, allowing each team member to become the subject matter expert (SME) for their component and assist with integration when issues arose. Beyond individual ownership, collaboration was maintained outside of scheduled meetings, with team members pulling down each other's code and testing using their own personal equipment. This combination of clear ownership and team-wide collaboration resulted in a working product.

Another strength was the technical approach to the system architecture. Building the ESP-NOW communication wrapper as a shared library allowed each sensor to be developed and tested independently before being integrated into the mesh. The custom message packet format proved to be a reliable and flexible foundation, staying well within the ESP-NOW 250-byte limit while carrying all necessary routing, health, and sensor data. The modular code design also made it straightforward to add new sensors without disrupting the existing network behavior.

6.0.2 Weaknesses. One significant area for improvement was the self-healing aspect of the project. The size of the task was underestimated, and there were no open-source MicroPython libraries available to handle self-healing mesh capabilities. The small testing environment also created challenges, as the ESP32's antenna range caused all nodes to default directly to the host, making it difficult to properly demonstrate multi-hop self-healing behavior. With more time, the team would recommend adopting a B.A.T.M.A.N. (Better Approach To Mobile Ad-hoc Networking) topology to address these gaps.

Another improvement could have been the process of the integration. The team handled integration very well given everyone was remote compared to the other person. The team lead Ian volunteered to setup a full mesh network in his apartment to conduct integration testing. While the team is proud of what was accomplished, multiple setups across the team could have led to faster integration while an ability to potentially add more to the overall system.

Another weakness is that the system still requires a technical understanding for an end user to set up and operate. Tasks such as manual device enrollment, direct editing of configuration files, and managing the host PC connection are not easy for a non-technical user to approach. While this was always intended as a proof of concept, the end result is not yet as user-friendly as off-the-shelf alternatives like Google Home [8] or Amazon Alexa or as organized as other custom options like Home Assistant[12]. This increases the barrier of entry for users seeking a fully private smart home solution without the willingness to tinker with the system.

6.1 Future Improvements

A key future improvement is the development of an automatic device-enrollment system. The current prototype requires users to manually enter device information and edit configuration files, which limits usability for non-technical users. A secure enrollment process with proper authentication would improve accessibility while strengthening network security.

The code base should also be further modularized to support future sensor expansion. A cleaner software structure would allow new nodes to be added with minimal changes to the existing communication framework. Secure remote access to the monitoring system would also improve usability by reducing the need for direct access to the host PC.

Future work should also improve network reliability through a more complete self-healing structure. RSSI measurements could be used to estimate signal strength between nodes and support a dynamic tree-based routing approach. In this structure, each node could select the strongest available parent node, improving transmission efficiency and reducing unnecessary retransmission.

Finally, low-power operation could be improved by extending deep-sleep periods for sensors that do not require frequent readings, while keeping interrupt-driven sensors idle until an event occurs.

7 POTENTIAL IMPACTS

7.0.1 Technical Domains. The team has come up with a prototype for an ESP Mesh network using commercial-off-the-shelf parts. This is a great starting point to building a more comprehensive

smart home to rival that of big technology companies like Apple, Google, and Amazon. With the system, we were able to build a fully customized smart home tailored to our needs, without having to clog network traffic running through the Wi-Fi.

7.0.2 Non-Technical Domains. From a non-technical standpoint, the system provides a distinctly different user experience from big tech solutions. Because it is fully standalone, users are not reliant on a vendor maintaining remote servers for the product to keep working. Once deployed, the system operates independently of any external service, meaning it cannot be rendered unusable by a company discontinuing support or experiencing an outage. Additionally, because the system is built on customizable ESP32 modules, users are not limited to the sensors or functionality offered by a particular manufacturer. Any node capable of interfacing with an ESP32 can be integrated into the network, giving users full control over the features and functionality of their smart home system.

8 CHANGE OF PLAN

The project plan was adjusted to place less focus on the GUI and greater emphasis on sensor and mesh network functionality. Since the GUI is more of a software concern than an embedded design challenge, it was determined that focusing on the embedded aspects of the project was better aligned with the overall goal. Efforts were redirected toward improving the quality of peer-to-peer communication, mesh design, and overall integration, with a terminal interface adopted in place of the originally proposed GUI.

Automatic enrollment of new sensors into the network was also set aside. Additionally, automatic device mapping was abandoned, as a consistent signal strength measurement method could not be established, making the development of an automatic mapping algorithm infeasible. As a result, each device requires manual direction from the user during setup.

Full self-healing network functionality was not achieved. Board-to-board route healing was demonstrated, but data packets were lost during the process, ultimately rendering the self-healing approach unusable in the final product.

9 EMBEDDED SYSTEMS ASPECTS

Embedded systems are task-specific and integrate both hardware and software, unlike general purpose computers. Each sensor was required to perform specific functions as part of an integrated system designed exclusively to function as a smart home. ESP32 micro controllers and MicroPython[11] were selected to provide a framework through which the sensors could communicate with each other.

GPIO pins, NeoPixel LEDs, and other hardware components were utilized on the ESP boards to physically interface with the sensors. I2C protocols were used to access sensor data, and baud rates were synced between sensors and boards to ensure correct data collection. Interrupts and callbacks synced with timers were implemented to facilitate the flow of information through the system without overloading the network.

ESP-NOW was selected as the embedded communication protocol for node-to-node data transfer, enabling a seamless mesh network. The peer-to-peer nature of ESP-NOW allowed nodes to

communicate directly without routing traffic through a Wi-Fi access point, which is a key distinction from general purpose networked systems. For the front door detection sensor, a Raspberry Pi was utilized as a dedicated camera node, capturing video input and passing detection results to an ESP32 node via UDP for relay across the mesh. The combination of these design decisions demonstrates that a true embedded system was built, rather than a general purpose computing device repurposed to handle these operations.

10 TASK BREAKDOWN

The project was divided among team members. Each engineer also supported and provided help as needed to others.

10.1 Ian

Ian setup the VPN network and host PC that everyone could remote into. He also lead up the weekly team meetings. He focused his engineering time on building a copy of every sensor and building the ESPnow communication wrapper.

10.2 John

John served as the co-lead and supported Ian with documentation and assignments. He also contributed to the ESP communication file. John also took on the stretch goal on building the self healing aspect of the mesh.

10.3 Majharul

Majharul headed up the design for battery health monitoring. He chose the module board to track battery health and node temperature. He also supported others in implementing the health monitoring into their sensors.

10.4 Alex

Alex built the room occupancy sensor. He laid the ground work for other sensors and provided early testing on the approach. He also worked on the logging and GUI.

10.5 Eugene

Eugene was assigned to work on the air quality sensors and helped with documentation of the project.

10.6 Johanan

Johanan was responsible for the raspberry pi person detection and the leak detection sensor. He also helped with GUI integration and with presentation slides.

11 RELATION TO OTHER PROJECT

Not applicable.

REFERENCES

- [1] Adafruit Industries LLC. [n. d.]. Liquid Level Sensor, Digital Output (Product No. 4965). DigiKey Electronics. <https://www.digikey.com/en/products/detail/adafruit-industries-llc/4965/14302510> Accessed: 2026-05-08.
- [2] Amazon.com, Inc. [n. d.]. Amazon Alexa Smart Home. <https://www.amazon.com/alexa-smart-home>. Accessed: 2026-05-08.
- [3] Analog Devices (Maxim Integrated). [n. d.]. MAX17048/MAX17049: 3 μ A 1-Cell/2-Cell Fuel Gauge with ModelGauge. <https://www.analog.com/media/en/technical-documentation/data-sheets/max17048-max17049.pdf> Accessed: 2026-05-08.

- [4] Aosong Electronics Co., Ltd. 2021. *DHT20 Humidity and Temperature Module Datasheet*. https://cdn-shop.adafruit.com/product-files/5183/5193_DHT20.pdf Accessed: 2026-05-08.
- [5] Apple Inc. [n. d.]. HomeKit — Apple Developer Documentation. <https://developer.apple.com/homekit/>. Accessed: 2026-05-08.
- [6] Espressif Systems. 2021. *ESP32-S3 Series Datasheet: 2.4 GHz Wi-Fi + Bluetooth® LE SoC*. https://cdn-shop.adafruit.com/product-files/5426/esp32-s3_datasheet_en.pdf Accessed: 2026-05-08.
- [7] Espressif Systems. 2024. *ESP-NOW FAQ*. <https://docs.espressif.com/projects/esp-faq/en/latest/application-solution/esp-now.html>
- [8] Google LLC. [n. d.]. Google Home. <https://home.google.com>. Accessed: 2026-05-08.
- [9] lady ada. 2025. *PIR Motion Sensor: Passive Infrared Proximity Motion Sensor*. Adafruit Industries. <https://learn.adafruit.com/pir-passive-infrared-proximity-motion-sensor> Accessed: 2026-05-08.
- [10] Wei Liu et al. 2016. SSD: Single Shot MultiBox Detector. In *European Conference on Computer Vision (ECCV)*.
- [11] MicroPython Contributors. [n. d.]. *MicroPython Documentation*. <https://docs.micropython.org/en/latest/index.html> Accessed: 2026-05-08.
- [12] Nabu Casa, Inc. [n. d.]. Home Assistant: Open Source Home Automation. <https://www.home-assistant.io>. Accessed: 2026-05-08.
- [13] Sensirion AG. 2022. *SGP40 Indoor Air Quality Sensor for VOC Measurements Datasheet*. https://sensirion.com/media/documents/296373BB/6203C5DF/Sensirion_Gas_Sensors_Datasheet_SGP40.pdf Accessed: 2026-05-08.
- [14] Bryan Siepert. 2024. *Adafruit BH1750 Ambient Light Sensor*. Adafruit Industries. <https://learn.adafruit.com/adafruit-bh1750-ambient-light-sensor> Accessed: 2026-05-08.
- [15] ThatOneGuyX1. 2025. ESP_SmartHome. https://github.com/ThatOneGuyX1/ESP_SmartHome. Accessed: 2026-05-07.